Introduction to Cybersecurity

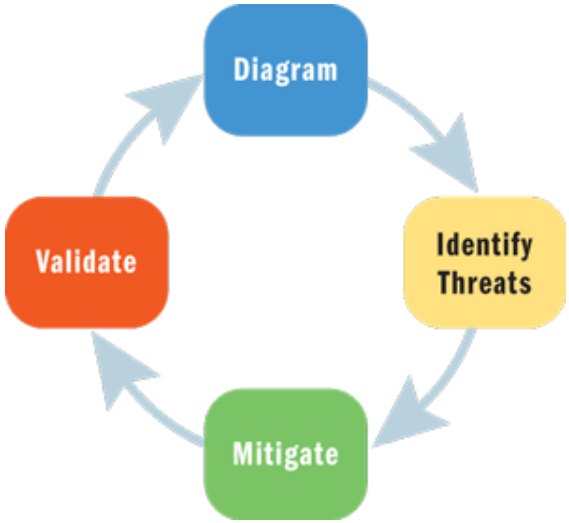
Threat Modeling



Jasmin Azemović
CISO | Associate Professor | Microsoft MVP

Summary

- Introduction
- Fundamentals of threat modeling



Introduction

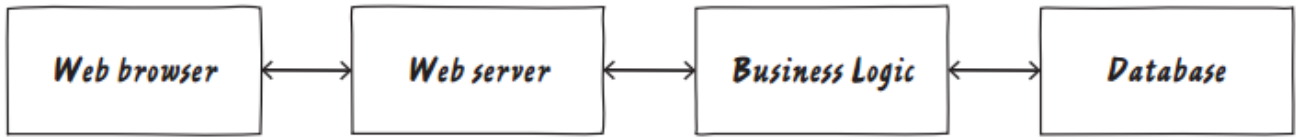
- Anyone can learn how to model a threat
- From an IT perspective and more than needed today
- Threat modeling is the process of applying a model to identify security issues
- The model provides abstraction without excess technical details
- A much broader picture of the system without it being complete

Fundamentals of threat modeling

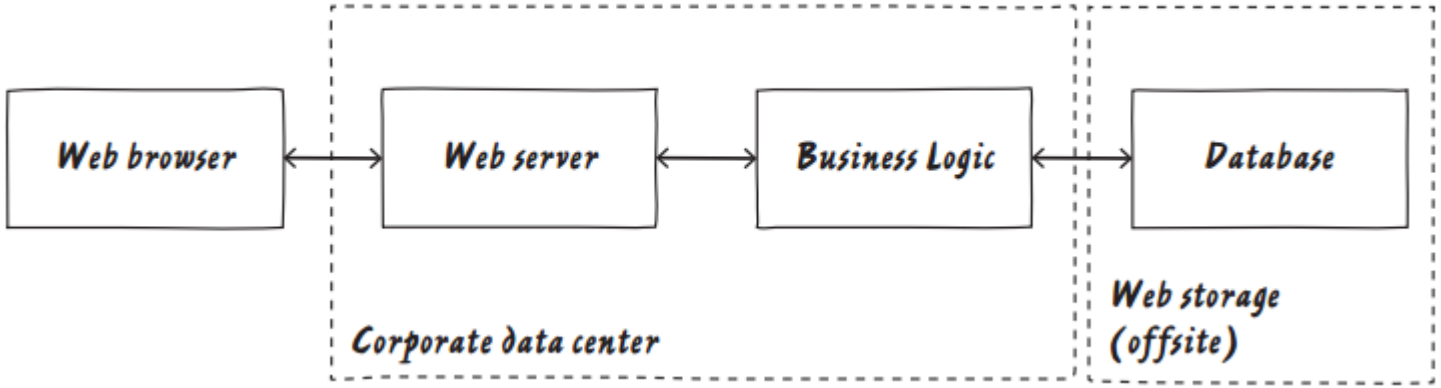
- What are we doing?
- Where can there be a problem?
- What can we do about the problem?
- STRIDE
- Examples

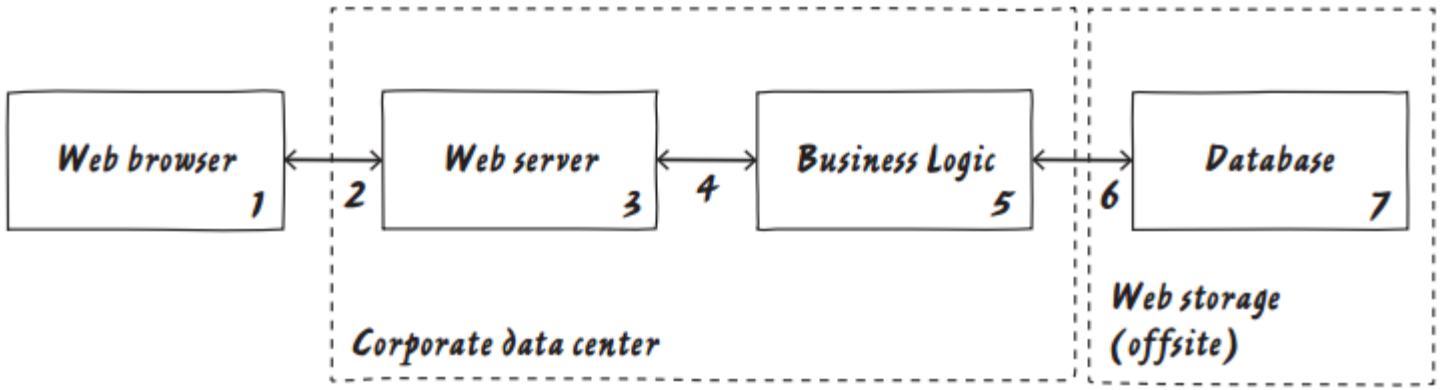
What are we doing?

- Diagrams are the best way to communicate during the development of any system
- Take for example a web application that consists of multiple components
- The first step is to identify "trust boundaries".
- A border is a space where other people or processes control different things



… what are we doing?





Where can there be a problem?

- Examples:
  - How do we know that the browser is being used by the right person?
  - What if the data in the database is modified?
  - Is the transfer between zones safe?
- These and similar threats can be identified by STRIDE pathways:
  - Spoofing
  - Tampering
  - Repudiation
  - Denial of Service
  - Information Disclosure
  - Elevation of Privilege
- Never ignore a threat, no matter how "small"

What can we do about the problem?

- There are four types of actions we can do:
- Mitigating
- Eliminating the threat
- Transferring
- Accepting risk

Spoofing

THREAT TARGET	MITIGATION STRATEGY	MITIGATION TECHNIQUE
Spoofing a person	Identification and authentication (usernames and something you know/have/are)	Usernames, real names, or other identifiers: ❖ Passwords ❖ Tokens ❖ Biometrics Enrollment/maintenance/expiry
Spoofing a "file" on disk	Leverage the OS	❖ Full paths ❖ Checking ACLs ❖ Ensuring that pipes are created properly
	Cryptographic authenticators	Digital signatures or authenticators
Spoofing a network address	Cryptographic	❖ DNSSEC ❖ HTTPS/SSL ❖ IPsec
Spoofing a program in memory	Leverage the OS	Many modern operating systems have some form of application identifier that the OS will enforce.

Tampering

THREAT TARGET	MITIGATION STRATEGY	MITIGATION TECHNIQUE
Tampering with a file	Operating system	ACLs
	Cryptographic	❖ Digital Signatures ❖ Keyed MAC
Racing to create a file (tampering with the file system)	Using a directory that's protected from arbitrary user tampering	ACLs Using private directory structures (Randomizing your file names just makes it annoying to execute the attack.)
Tampering with a network packet	Cryptographic	❖ HTTPS/SSL ❖ IPsec
	Anti-pattern	Network isolation (See note on network isolation anti-pattern.)

Repudation

THREAT TARGET	MITIGATION STRATEGY	MITIGATION TECHNIQUE
No logs means you can't prove anything.	Log	Be sure to log all the security-relevant information.
Logs come under attack	Protect your logs.	❖ Send over the network. ❖ ACL
Logs as a channel for attack	Tightly specified logs	Documenting log design early in the development process

Information Disclousure

THREAT TARGET	MITIGATION STRATEGY	MITIGATION TECHNIQUE
Network monitoring	Encryption	❖ HTTPS/SSL ❖ IPsec
Directory or filename (for example `layoff-letters/adamshostack.docx`)	Leverage the OS.	ACLs
File contents	Leverage the OS.	ACLS
	Cryptography	File encryption such as PGP, disk encryption (FileVault, BitLocker)
API information disclosure	Design	Careful design control Consider pass by reference or value.

Denial of Service

THREAT TARGET	MITIGATION STRATEGY	MITIGATION TECHNIQUE
Network flooding	Look for exhaustible resources.	❖ Elastic resources ❖ Work to ensure attacker resource consumption is as high as or higher than yours.
		Network ACLS
Program resources	Careful design	Elastic resource management, proof of work
	Avoid multipliers.	Look for places where attackers can multiply CPU consumption on your end with minimal effort on their end: Do something to require work or enable distinguishing attackers, such as client does crypto first or login before large work factors (of course, that can't mean that logins are unencrypted).
System resources	Leverage the OS.	Use OS settings.

Elevation of Privilege

THREAT TARGET	MITIGATION STRATEGY	MITIGATION TECHNIQUE
Data/code confusion	Use tools and architectures that separate data and code.	❖ Prepared statements or stored procedures in SQL ❖ Clear separators with canonical forms ❖ Late validation that data is what the next function expects
Control flow/ memory corruption attacks	Use a type-safe language.	Writing code in a type-safe language protects against entire classes of attack.
	Leverage the OS for memory protection.	Most modern operating systems have memory-protection facilities.

Examples

- General examples might look like this:
  - A thief who wants to steal money
  - Company employees who have access to sensitive documents
- IT examples
  - Insecure network
  - An attacker who wants to steal a browser session (cookie)
  - An attacker who wants to read and change data
    - No traces
- We need to have a clear answer to this question
- Failure to do so may result in inconsistency and loss of resources

Resources

- https://www.mistral.ba/learn-to-threat-model/
- https://dr-knz.net/stride-threat-model-with-examples.html
- https://www.amazon.com/Threat-Modeling-Designing-Adam-Shostack/dp/1118809998
- https://www.g2.com/articles/threat-modeling

